

06 — Developer & Deployment

Complete documentation suite (Markdown → PDF with Mermaid diagrams)

Generated: 2026-09-22

Source: repository docs/ directory

Diagrams rendered with Mermaid 11

Table of Contents

- 06-developer-deployment/01-development-overview.md
- 06-developer-deployment/02-prerequisites.md
- 06-developer-deployment/03-local-development.md
- 06-developer-deployment/04-project-structure.md
- 06-developer-deployment/05-environment-variables.md
- 06-developer-deployment/06-configuration.md
- 06-developer-deployment/07-running-the-application.md
- 06-developer-deployment/08-testing.md
- 06-developer-deployment/09-code-quality.md
- 06-developer-deployment/10-git-workflow.md
- 06-developer-deployment/11-docker.md
- 06-developer-deployment/12-deployment.md
- 06-developer-deployment/13-production-runbook.md
- 06-developer-deployment/14-monitoring.md
- 06-developer-deployment/15-logging.md
- 06-developer-deployment/16-backup-and-restore.md
- 06-developer-deployment/17-disaster-recovery.md
- 06-developer-deployment/18-troubleshooting.md
- 06-developer-deployment/19-security.md
- 06-developer-deployment/20-performance.md

Development Overview

OPMS is a polyglot-free **Node.js + Next.js monorepo** orchestrated with Docker Compose.

Typical developer loop

1. Configure `.env.docker` from example (use placeholders for secrets)
2. Start stack with compose (optionally `docker-compose.dev.yml` for hot reload)
3. Open App Frontend `:7013` or OPMS `:7002`
4. Use Swagger at `:7001/api-docs` for OPMS API exploration

Packages

13 `package.json` projects (8 backends + 5 frontends). No root workspace package manager identified.

Related

- Local Development
- DOCKER.md

Prerequisites

Tool	Purpose	Evidence
Docker Desktop / Engine + Compose v2	Run full stack	DOCKER.md
Node.js 20	Image base <code>node:20-alpine</code> ; local non-docker optional	Dockerfiles
Git	Source control	<code>.git</code>
MongoDB Atlas or local Mongo	Database	<code>MONGO_URI</code>
Optional: host nginx + certbot	TLS prod	<code>nginx/</code>

Not required for basic Docker run

Kubernetes, RabbitMQ, local MinIO (file API external).

Local Development

Steps (verified from DOCKER.md)

1. Clone repository
2. `cp .env.docker.example .env.docker`
3. Edit `.env.docker`: set `JWT_SECRET`, `MONGO_URI` / `MONGODB_URI` (**do not commit secrets**)
4. `ln -sf .env.docker .env`
5. Start:
 - Prod-style: `docker compose --env-file .env.docker up --build`
 - Dev hot reload: `docker compose --env-file .env.docker -f docker-compose.yml -f docker-compose.dev.yml up --build`
6. Verify: `http://localhost:7001/health` and App Frontend `http://localhost:7013`
7. Optional seed: `docker compose --env-file .env.docker exec opms-backend npm run seed:users`

Per-service local run (without compose)

Possible via each package `npm run dev / start` if env vars and Mongo/Redis are provided — exact standalone runbooks **partially documented**; prefer Compose.

URLs

See DOCKER.md table (7001–7014).

Project Structure

```
/
├─ app-frontend/           # SSO hub Next.js
├─ opms-frontend/         # OPMS UI
├─ user-manager-frontend/
├─ lead-manager-frontend/
├─ work-planner-frontend/
├─ auth-service/
├─ opms-backend/          # BFF + order domain
├─ product-service/
├─ party-service/
├─ lead-manager-backend/
├─ work-planner-backend/
├─ message-service/
├─ notification-service/
├─ nginx/                 # Host TLS reverse proxy configs
├─ docker-compose.yml
├─ docker-compose.dev.yml
├─ docker-compose.prod.yml # Partial stack
├─ DOCKER.md
├─ .env.docker.example
├─ docs/                  # This documentation suite
├─ backups/               # gitignored leftovers
└─ minio-data/            # leftover, not compose service
```

Typical backend layout

```
src/server.js , src/app.js , src/modules/ , src/models OR src/data/mongoRegistry.js , src/middlewares/ ,
src/config/
```

Environment Variables

IMPORTANT: Never copy real secrets into documentation.

SECRET DETECTED IN SOURCE — NOT INCLUDED IN DOCUMENTATION: `.env.docker.example` currently contains values that look like live JWT secrets, Mongo credentials, API keys, and passwords. Treat as a credential-hygiene incident; rotate if exposed; replace example file with placeholders.

Variable	Required	Purpose	Example (placeholder)	Secret
NODE_ENV	yes	Runtime mode	production	no
JWT_SECRET	yes	JWT signing	change-me-long-random	yes
JWT_EXPIRES_IN	no	Token TTL	7d	no
MASTER_PASSWORD	no	Break-glass login	empty in prod	yes
JSON_BODY_LIMIT	no	Body size	50mb	no
MONGO_URI / MONGODB_URI	yes	Mongo connection	mongodb+srv://USER:PASS@host/db	yes
MONGODB_LOOKUP_FAMILY	no	Prefer IPv4	4	no
REDIS_URL	no	Redis	redis://redis:6379	no*
REDIS_PORT	no	Host publish	7014	no
*_PORT	no	Host ports	see compose defaults	no
AUTH_SERVICE_URL etc.	no	Internal URLs	http://auth-service:7003	no
API_PUBLIC_BASE_URL	prod	Public API	https://api-opms.example.com	no
NEXT_PUBLIC_*	prod	Browser URLs (build-time)	https://...	no
CORS_ORIGINS	prod	Extra CORS	comma URLs	no
COMPANY_NAME / LOGO	no	Branding fallback	—	no
SEED_*	no	Seed controls	—	SEED_PASSWORD yes
FOLLOWUP_REMINDER_*	no	Lead cron	—	no
FILE_MANAGEMENT_API_URL/KEY	optional	Uploads	—	key yes
SMTP_*	optional	Email	—	pass yes
MICROSOFT_GRAPH_*	optional	Email	—	secret yes
WHATSAPP_*	optional	WhatsApp	—	yes

Variable	Required	Purpose	Example (placeholder)	Secret
VAPID_*	optional	Web push	—	private yes
GOOGLE_SHEET_WEBHOOK_SECRET	optional	Webhooks	—	yes
ADMIN_EMAIL / ACCOUNT_EMAIL / FINANCE_EMAIL / DUE_SHEET_EMAIL / DISPATCH_EMAIL	optional	Role mailboxes	—	no
COMPOSE_PROJECT_NAME / ENV_FILE	no	Multi-stack	—	no
APP_LOGIN_URL / LEAD_MANAGER_PUBLIC_URL / FRONTEND_URL	optional	Links in emails	—	no

* Redis currently unauthenticated in compose.

Source: `.env.docker.example` , service `env.js` files, compose.

Configuration

Patterns

- dotenv / compose `env_file`
- Service `src/config/env.js` reading `process.env`
- Frontend `NEXT_PUBLIC_*` baked at **image build**

Do not

Re-declare `MONGO_URI` / `JWT_SECRET` under compose `environment:` with empty interpolation — overrides `env_file` (DOCKER.md).

Nginx

Copy snippets + site files to `/etc/nginx/` ; use HTTP bootstrap before certbot; then HTTPS site.

Running the Application

Full stack

```
docker compose --env-file .env.docker up --build
# or detached
docker compose --env-file .env.docker up -d --build
```

Dev hot reload

```
docker compose --env-file .env.docker -f docker-compose.yml -f docker-compose.dev.yml up --build
```

Partial prod file (incomplete)

```
docker compose --env-file .env.docker -f docker-compose.prod.yml up --build -d
```

Warning: missing many services vs full product.

Useful

```
docker compose --env-file .env.docker logs -f opms-backend
docker compose --env-file .env.docker down
docker compose --env-file .env.docker down -v # wipes Redis volume
docker compose --env-file .env.docker exec opms-backend npm run seed:users
```

Package scripts (examples)

- Backends: `npm start`, `npm run dev`
- Frontends: `npm run dev`, `npm run build`, `npm start`, `npm run lint`
- opms-backend also has migrate/seed scripts in `package.json`

Testing

Status

Not identified as a comprehensive automated test suite at monorepo root (no Jest/Playwright configs systematically documented across all packages in discovery).

What exists

- Lint scripts on frontends (`npm run lint`)
- Health endpoints for smoke checks
- Manual verification via UI + Swagger

Recommendation

Requires confirmation from engineering on intended test strategy; do not invent coverage claims.

Code Quality

Mechanism	Where
ESLint via <code>next lint</code>	Frontends package scripts
Swagger for API surface	opms-backend <code>/api-docs</code>
Soft-delete plugin consistency	Shared plugin copies

No monorepo-wide prettier/eslint root config identified as mandatory CI gate (no CI).

Git Workflow

Observed

- Git repository at monorepo root
- `.gitignore` excludes `.env.docker` , backups, minio-data, etc.

Formal branching / PR policy

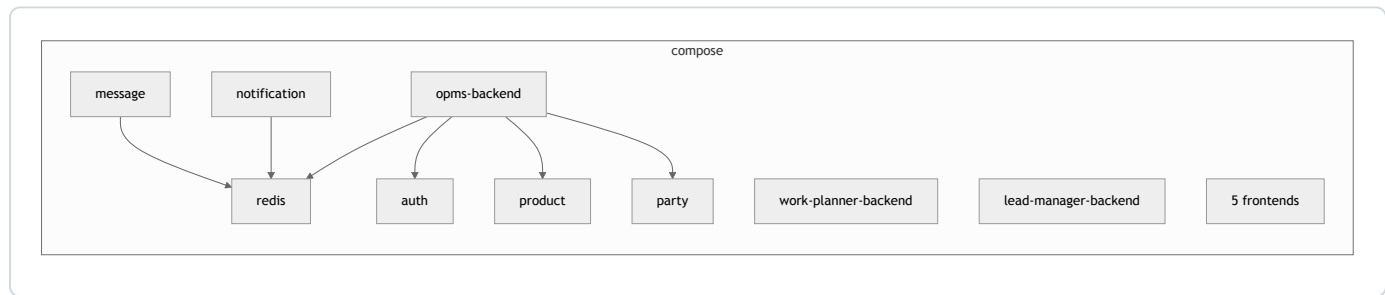
Not identified in repository (no CONTRIBUTING.md).

Requires confirmation from team process.

Docker

Compose services (base)

redis + 8 APIs + 5 frontends. See [Infrastructure Architecture](#).



Images

Build contexts per service folder; `node:20-alpine` .

Volumes

`redis_data` only.

Healthchecks

Backends + redis; frontends none in base compose.

Dev overlay

`Dockerfile.dev` , bind-mount `src` , watch modes, FE healthchecks disabled.

Related

Root [DOCKER.md](#)

Deployment

Path A — Single host Docker + nginx (documented)

1. Provision host with Docker + nginx + certbot
2. Configure `.env.docker` with production URLs and secrets
3. `docker compose --env-file .env.docker up -d --build`
4. Install nginx site (`nginx/sites-available/opms`) and obtain certs (commands in nginx file headers)
5. DNS for all hostnames
6. Verify `/health` on APIs and FE loads

Path B — Medica tenant

Use `.env.docker.medica` + medica nginx; ports 5001–5013. Dual-stack on one host **Requires confirmation**.

Image registry / K8s

Not identified.

Rollback

Rebuild/redeploy previous git SHA; no automated rollback tool in repo.

Production Runbook

Startup

```
docker compose --env-file .env.docker up -d
```

Health verification

```
curl -fsS http://127.0.0.1:7001/health
curl -fsS http://127.0.0.1:7003/health
# repeat for other API ports
```

Logs

```
docker compose --env-file .env.docker logs -f <service>
```

Restart one service

```
docker compose --env-file .env.docker restart opms-backend
```

Seed / admin bootstrap

Seed scripts + User Manager to assign portals. Exact first-admin procedure **Requires confirmation**.

Incidents

See Troubleshooting.

Monitoring

Capability	Status
/health endpoints	Implemented on backends
Redis healthcheck	Compose redis-cli ping
Prometheus/Grafana/APM	Not identified
Uptime checks	Not identified in repo
Alerting	Not identified

SSE streams exist for notifications — not a metrics system.

Logging

- Application logs via container stdout/stderr
- Access via `docker compose logs`
- No ELK/Loki stack in compose

Structured logging standard: **Not identified** as global.

Backup and Restore

What needs backup

Data	Location	Repo automation
MongoDB	Atlas/external	Not identified
Redis	<code>redis_data</code> volume	Not identified (often ephemeral queues)
Files	External File API / MinIO	Not identified
Config	<code>.env.docker</code> (secrets store outside git)	operator-managed
nginx certs	host <code>/etc/letsencrypt</code>	operator-managed

Observed artifacts

`backups/minio-backup-*.tar.gz` leftover; `dump.rdb` gitignored — **not** a documented procedure.

Commands

Not identified official backup/restore scripts. Use MongoDB Atlas backups / `mongodump` per ops standards — **Requires confirmation.**

Disaster Recovery

Status

No formal DR runbook in repository.

Inferred recovery outline (not official)

1. Restore Mongo from Atlas PITR/snapshot
2. Redeploy compose stack from known-good git tag
3. Restore env secrets from secure vault
4. Re-point DNS/nginx if host replaced
5. Verify login + create-order smoke test

Requires confirmation from ops for RTO/RPO targets.

Troubleshooting

Compose won't start / env overrides

Symptoms: Mongo auth failures despite `env_file`

Cause: Empty `${MONGO_URI}` in `environment:` overrides file

Solution: Follow DOCKER.md — don't redeclare secrets in environment

Verify: `docker compose exec auth-service printenv MONGO_URI` (ensure set; don't log in shared chats)

JWT errors across services

Symptoms: 401 after login when calling another service

Cause: Mismatched `JWT_SECRET`

Solution: Single shared secret in `.env.docker`

Verify: Relogin; call `/api/auth/me`

403 on OPMS routes

Symptoms: Authenticated but forbidden

Cause: Missing `portals[]` entry `portal_code: opms`

Solution: Assign portal in User Manager; re-login

Verify: `/api/auth/me` shows portals

Frontend calls wrong API / CORS

Symptoms: Network errors to localhost from prod build

Cause: `NEXT_PUBLIC_*` baked at build

Solution: Fix env and **rebuild** frontend images

Verify: View page source / network hostnames

Redis / queue jobs stuck

Symptoms: Notifications/emails not sending

Cause: Redis down or workers not started

Solution: Check redis health; restart message/notification/opms-backend

Verify: `docker compose ps` ; service logs

Mongo Atlas IPv6 issues

Symptoms: Connection timeouts from containers

Cause: DNS family

Solution: `MONGODB_LOOKUP_FAMILY=4`

Verify: Backend connects and `/health` OK

File upload failures

Symptoms: Attachment create errors

Cause: Missing `FILE_MANAGEMENT_API_*`

Solution: Configure URL/key or disable upload features

Verify: Upload small file in UI

nginx 502

Symptoms: Public hostname 502

Cause: Container not listening / wrong upstream port

Solution: Align nginx upstream with compose ports; `nginx -t` + reload

Verify: curl upstream on 127.0.0.1:port

Security

Implemented

- JWT auth + bcrypt passwords
- Portal RBAC
- HTTPS via nginx TLS configs
- Webhook secrets for Google Sheets
- Soft-delete vs hard-delete for many records

Risks identified in codebase (document, do not "fix" here)

Risk	Notes
Secrets in <code>.env.docker.example</code>	Appears to contain real credentials — rotate & sanitize
<code>MASTER_PASSWORD</code>	Break-glass; must be empty in prod
Redis without AUTH + published port	Network exposure risk
Notification internal routes unauthenticated	Network trust assumed
opms terms-and-conditions router without <code>requireAuth</code>	Exposure risk
lead-manager attachments router without <code>requireAuth</code>	Exposure risk
work-planner attachment view before auth	Intentional public view? Requires confirmation
Duplicated schema registries	Drift / authz inconsistency risk

Rate limiting

Not identified.

Related

- Auth architecture

Performance

Config levers found

- `JSON_BODY_LIMIT`
- nginx proxy timeouts / body size for API hosts
- Redis/BullMQ async offload for messages/notifications/some order jobs
- Mongo indexes on hot fields

Targets

No published latency/throughput SLOs in repo.

Frontend

Next.js production builds; `NEXT_PUBLIC_*` must be correct to avoid extra failures (not perf per se).